

Highlights

CAGE-S: Compliance-Gated Online Scheduling with Temporal Risk Prediction and Pre-Commit Guardrails for Sensitive Edge–Cloud Microservices

Anonymous Author(s)

- A legality-first scheduler integrates policy, temporal risk, and guardrails.
- Temporal history predicts unsafe next states better than current telemetry alone.
- Fair shortlisting preserves decision quality with millisecond-scale overhead.
- Zero policy, temporal, and Q95 violations occur in 462,480 seeded decisions.

CAGE-S: Compliance-Gated Online Scheduling with Temporal Risk Prediction and Pre-Commit Guardrails for Sensitive Edge–Cloud Microservices

Anonymous Author(s)^a

^aAnonymous institution, Anonymous country

ARTICLE INFO

Keywords:

Edge–cloud computing
Microservice scheduling
Policy-aware placement
Temporal risk prediction
Pre-commit guardrail
Kubernetes

ABSTRACT

Sensitive Internet-of-Things services increasingly execute as microservices across edge, fog, and cloud nodes that differ not only in capacity, but also in jurisdiction, trust level, encryption support, and hardware isolation. A useful scheduler must therefore exclude illegal placements, anticipate near-term overload, avoid repeatedly concentrating decisions on a small set of nodes, and verify the selected destination immediately before commitment. Existing schedulers usually address these concerns separately: policy engines enforce legality without predicting operational risk, while performance-oriented methods can select a destination that is currently feasible but becomes overloaded in the next control interval.

This paper presents CAGE-S, a legality-first online scheduler that combines deterministic policy gating, a calibrated temporal-risk model, fairness-aware candidate shortlisting, and an empirical Q95 pre-commit guardrail. The method is evaluated through trace-driven counterfactual replay using the Alibaba Microservices Trace 2021. After strict cleaning and temporal alignment, the evaluation contains 20,324 events over 12,727 nodes and 108 decision timestamps. The temporal histogram-gradient-boosting model achieves a test ROC-AUC of 0.99825, PR-AUC of 0.99031, and Brier score of 0.00858. On the natural test workload, CAGE-S migrates 39.12% of events while producing zero policy, temporal-threshold, and Q95 guardrail violations; the selected native next-state unsafe proxy is 0.043%. Relative to a matched current-state risk scheduler, it reduces migration by 3.73 percentage points and removes a 0.370% temporal-violation rate. Across 30 deterministic policy seeds and 462,480 decisions, no policy violation, guardrail violation, destination conflict, or fallback is observed. The measured online core requires 20.25 ms on average for the natural workload and 13.15 ms for the balanced workload. Paired timestamp-cluster bootstrap analysis confirms the principal trade-offs. The native next-state metric is interpreted strictly as a trace proxy, not as a causal estimate of post-migration performance.

1. Introduction

Microservice architectures are now common in Internet-of-Things (IoT) platforms because they allow sensing, preprocessing, inference, storage, and control functions to evolve independently and to be placed across the edge–cloud continuum. This flexibility is valuable when an application must react close to a device while still using elastic cloud resources. It also makes placement decisions harder. A scheduler is no longer choosing among interchangeable servers in one trusted data centre. It may be choosing among nodes in different legal regions, nodes operated by different providers, and nodes with different encryption, attestation, and isolation capabilities. At the same time, those nodes experience bursty and strongly autocorrelated load. A node that appears acceptable at the instant of observation can become a poor destination one control interval later.

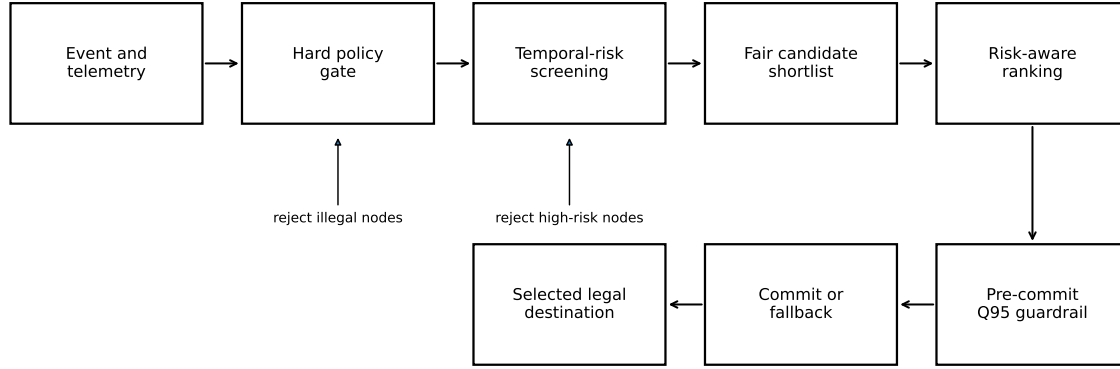
Most scheduling work has focused on performance. Network-aware systems place communicating services close together, learning-based schedulers adapt to heterogeneous resources, and SLO-oriented controllers react to latency or resource contention (Shen, Han, Wang, Wang and Leung, 2023; Zhu, Zhu et al., 2023; Chen, Islam, Rodriguez Read and Buyya, 2026; Qiu, Banerjee, Jha, Kalbarczyk and Iyer, 2020). These systems answer an important question: where

can a service run efficiently? Security and governance work answers a different question: where is the service allowed to run? Policy-as-Code frameworks, trust-aware enrolment, node affinity, and confidential-computing labels can enforce jurisdiction and capability requirements (Pallewatta and Babar, 2024; Thijsman, Sebrechts, De Turck and Volckaert, 2024). In practice, a sensitive IoT scheduler must answer both questions in one decision path.

Treating compliance as a penalty inside a performance objective is unsafe. A sufficiently attractive performance score can compensate for the penalty and make an illegal node look optimal. Filtering illegal nodes only after ranking is also problematic because the optimizer spends effort on candidates that can never be committed, and the fallback behaviour is often unspecified. We therefore adopt a legality-first design: a node that does not satisfy the service policy never enters the optimization set. This design choice is simple, but it changes the semantics of the scheduler. Compliance is a hard precondition rather than one more tunable objective.

Legality alone is not enough. Static CPU and memory thresholds detect obvious overload, but they ignore direction and persistence. Two nodes with the same current utilization can have very different near-term risk if one has been stable and the other is rising rapidly. Recent telemetry-driven resource managers show that short temporal context is useful for SLO management, anomaly localization, and elastic

ORCID(s):



Legality is enforced before optimization; temporal risk and the empirical Q95 projection are complementary filters.

Figure 1: The CAGE-S decision path. Hard policy gating precedes all optimization. Temporal-risk screening and the empirical Q95 projection serve different purposes: the former predicts near-term threshold crossing from history, while the latter provides a transparent pre-commit headroom check.

scaling (Qiu et al., 2020; Gan, Liang, Dev, Lo and Delimitrou, 2021; Wen, Xu, Gill, Hilman, Srirama, Ye and Xu, 2024). However, temporal prediction has rarely been integrated with hard policy gating and a commit-time resource guardrail in a single online placement pipeline.

A further difficulty is concentration. When many decisions are made at the same timestamp, repeatedly selecting the lowest predicted-risk node can create a new hotspot or make a small group of nodes carry most migrations. A full fairness optimization across thousands of nodes at every event is unnecessary, but ignoring historical selection counts is equally undesirable. The scheduler in this paper first constructs a fairness-aware shortlist and then ranks the shortlisted legal and low-risk nodes. This preserves diversity without turning the problem into a large multi-objective search.

This paper presents CAGE-S, short for *Compliance-Aware Guardrail-Enforced Scheduling*. The method has four layers. A deterministic policy gate removes nodes that do not satisfy jurisdiction, trust, encryption, and trusted-execution requirements. A calibrated temporal classifier estimates the probability that a node will cross an operational utilization threshold in the next 30-second interval. A fairness-aware shortlist limits candidate evaluation while preserving low-use destinations. Finally, an empirical Q95 projection checks CPU and memory headroom immediately before the selection is committed. Figure 1 summarizes the decision path.

The evaluation uses the Alibaba Microservices Trace 2021 (Luo, Xu, Lu, Ye, Xu, Zhang, Ding, He and Xu, 2021). We align microservice events with node telemetry so that the scheduler sees the state at event time t and the outcome proxy is measured at $t + 30$ seconds. This alignment is important: using the destination’s future state at t or building temporal

features from future samples would leak information. The resulting evaluation contains 20,324 events and 12,727 nodes. Because the trace does not contain jurisdiction or sensitivity labels, the policy overlay is deterministic, independent of telemetry and outcomes, and evaluated over 30 seeds. The trace also does not reveal the true counterfactual effect of migrating a microservice to a different node. We therefore call the measured future-state outcome a *native next-state proxy*; it is useful for controlled comparison, but it is not presented as causal evidence.

The main contributions are as follows.

- (i) We formulate sensitive microservice placement as a legality-first online scheduling problem in which policy feasibility, temporal operational risk, fairness, and commit-time executability have explicit and non-interchangeable roles.
- (ii) We design a practical scheduler that combines deterministic policy masks, a 22-feature temporal histogram-gradient-boosting model, fairness-aware k -shortlisting, and an empirical Q95 guardrail. The complete online core is deterministic and supports explicit fallback and reservation checks.
- (iii) We build a leakage-controlled trace-replay protocol from the Alibaba 2021 microservice trace, including strict cleaning, temporal alignment, a natural workload, a service-balanced workload, and 30-seed policy sensitivity analysis.
- (iv) We provide an extensive comparison against current-state prediction, static Q95 and boundary rules, random and least-loaded legal placement, policy-only

scheduling, temporal-only scheduling, and ablated variants. Paired timestamp-cluster bootstrap analysis quantifies the trade-offs rather than claiming universal dominance.

The remainder of the paper is organized as follows. Section 2 reviews scheduling, temporal resource management, and policy-aware edge orchestration. Section 3 defines the system model and problem. Section 4 presents CAGE-S. Sections 5 and 6 describe the trace protocol and results. Section 7 discusses novelty, trade-offs, limitations, and practical deployment, and Section 8 concludes the paper.

2. Related work

2.1. Dynamic microservice scheduling

Kubernetes provides extensible filtering and scoring interfaces, but its default scheduler does not continuously reconsider placements when traffic, network delay, or service-level behaviour changes (Kubernetes Authors, 2026). This gap has motivated network-aware and adaptive schedulers. NetMARKS uses service-mesh telemetry to score nodes according to traffic between the target pod and pods already placed on each node (Wojciechowski, Opasiak, Latusek, Wereski, Morales, Kim and Hong, 2021). Network-aware Kubernetes extensions additionally incorporate infrastructure delay and service interaction information (Marchese and Tomarchio, 2022b,a). OptTraffic optimizes traffic scheduling among multi-replica containerized microservices (Zhu et al., 2023), while ChainsFormer provisions resources using chain-level latency information (Song, Xu, Ye, Wu, Gill, Buyya and Xu, 2023).

Recent work moves beyond one-shot placement. KaiS coordinates request dispatch and service orchestration through graph representation learning and multi-agent reinforcement learning in Kubernetes-oriented edge-cloud networks (Shen et al., 2023). ConAdapter uses reinforcement learning to adapt microservice concurrency quickly under changing demand (Liu, Zhang and Wang, 2023). TraDE, published in IEEE Transactions on Parallel and Distributed Systems, reconstructs bidirectional traffic stress and a node-delay matrix, then uses a parallel greedy mapper to redeploy microservices after a QoS violation (Chen et al., 2026). These methods are highly relevant to dynamic scheduling, but their main objectives are throughput, communication cost, or response time. They do not make data-governance legality a first-class gate, and they generally do not combine a calibrated next-interval node-risk predictor with a commit-time empirical guardrail.

2.2. SLO-aware resource management and temporal prediction

Telemetry-driven resource management has shown that microservice behaviour cannot be understood from a single instantaneous sample. FIRM localizes SLO violations, identifies contended resources, and applies fine-grained reprovisioning (Qiu et al., 2020). Sage models service dependencies to diagnose performance problems and recover

QoS (Gan et al., 2021). GrandSLAM allocates latency budgets across stages in a microservice execution framework (Kannan, Subramanian, Raju, Ahn, Mars and Tang, 2019), and Parslo derives partial SLO allocations through gradient-based optimization (Mirhosseini, Elnikety and Wenisch, 2021). StatuScale uses status detection to select elastic scaling actions under spikes and bursts (Wen et al., 2024). Resource-management work on shared microservices has also emphasized scalable modelling of calling graphs and interference (Luo, Lin, Ye, Xu, Zhang, Yang, Xu and Xu, 2024).

These systems establish the value of history, dependency structure, and online telemetry. The distinction in CAGE-S is architectural. Temporal prediction is not an isolated forecasting task and is not allowed to override policy. It is one gate in a placement pipeline whose final action must still satisfy a transparent pre-commit headroom test. The evaluation also separates model discrimination from scheduler-level outcomes: a higher ROC-AUC is not treated as sufficient evidence unless it changes placement behaviour under matched operating conditions.

2.3. Policy, trust, and secure edge-cloud management

Sensitive IoT applications introduce requirements that are not captured by resource capacity alone. Data may need to remain in a jurisdiction, execute only on nodes above a trust level, use encrypted transport, or require a trusted execution environment. Pallewatta and Babar propose a Policy-as-Code architecture for secure management of edge-cloud IoT microservices using Kubernetes, Istio, and Open Policy Agent (OPA) (Pallewatta and Babar, 2024). Remote-attestation work similarly demonstrates that Kubernetes edge workers can be enrolled and authorized according to hardware-rooted trust evidence (Thijsman et al., 2024). The broader placement literature identifies security, privacy, locality, heterogeneity, and mobility as recurring constraints for fog-hosted IoT microservices (Pallewatta, Kostakos and Buyya, 2023).

Policy engines are strong at deciding whether a candidate is permitted, but they do not by themselves predict whether the candidate will cross a resource threshold in the next scheduling interval. Conversely, a learned performance model can rank nodes accurately while remaining unaware of jurisdiction or trust. CAGE-S connects these layers without weakening either one: policy determines the candidate universe; prediction and fairness order the legal candidates; the guardrail verifies the selected action.

2.4. Positioning of this work

Table 1 summarizes the closest lines of work. No single cited system jointly provides hard legality filtering, temporal node-risk prediction, fairness-aware destination selection, and an explicit pre-commit projection. The novelty is not a new classifier in isolation. It is the separation and integration of four decision responsibilities that are often conflated: *permission, near-term risk, distribution of repeated decisions,*

Table 1

Positioning against representative related systems. A check mark indicates an explicit component of the published method, not a capability that might be added externally.

System	Dynamic rescheduling	Policy gate	Temporal risk	Fair destination use	Pre-commit projection	Main optimization signal
NetMARKS (Wojciechowski et al., 2021)	✗	✗	✗	✗	✗	Service-mesh traffic score
KaiS (Shen et al., 2023)	✓	✗	learned policy	indirect	✗	Long-term request throughput
OptTraffic (Zhu et al., 2023)	✓	✗	✗	✗	resource feasibility	Cross-machine traffic
TraDE (Chen et al., 2026)	✓	✗	QoS trigger	load penalty	capacity penalty	Traffic-delay communication cost
Policy-as-Code (Pallewatta and Babar, 2024)	✓	✓	✗	✗	policy recheck	Security-policy enforcement
CAGE-S	✓	✓	✓	✓	✓	Lexicographic legality, fairness, risk, and Q95 headroom

and *commit-time executability*. This separation makes the decision path easier to audit and prevents performance scores from compensating for policy violations.

3. System model and problem formulation

3.1. Scheduling setting

Let $\mathcal{N} = \{n_1, \dots, n_M\}$ be the available edge, fog, and cloud nodes, and let $\mathcal{S} = \{s_1, \dots, s_Q\}$ be the microservices. Decisions occur at discrete control timestamps $t \in \mathcal{T}$ separated by 30 seconds in the trace replay. An event $e = (s_e, n_e, t)$ identifies the service instance, its observed source node n_e , and the decision timestamp. Multiple events may occur at the same timestamp.

Each node has an operational state

$$\mathbf{x}_{n,t} = [c_{n,t}, m_{n,t}] \in \mathbb{R}^2, \quad (1)$$

where $c_{n,t}$ and $m_{n,t}$ are the observed CPU and memory utilization signals. The values are treated as node-level telemetry features. Instance-level CPU and memory values in the Alibaba shard are not summed into absolute node capacity because the trace does not provide a compatible accounting basis; preliminary analysis showed that such aggregation would be misleading.

A node also has a capability profile

$$\mathbf{a}_n = (j_n, \tau_n, e_n, z_n), \quad (2)$$

where j_n is jurisdiction, τ_n is trust rank, $e_n \in \{0, 1\}$ indicates encryption support, and $z_n \in \{0, 1\}$ indicates trusted-execution support. A service has a requirement profile

$$\mathbf{r}_s = (J_s, \tau_s^{\min}, e_s^{\min}, z_s^{\min}). \quad (3)$$

The legality predicate is

$$L(s, n) = \mathbb{I}[j_n \in J_s] \mathbb{I}[\tau_n \geq \tau_s^{\min}] \mathbb{I}[e_n \geq e_s^{\min}] \mathbb{I}[z_n \geq z_s^{\min}]. \quad (4)$$

If $L(s, n) = 0$, the node is never scored for that service.

3.2. Next-interval operational risk

A node is defined as operationally unsafe at $t + 1$ when either observed utilization exceeds the threshold $\rho = 0.8$:

$$y_{n,t+1} = \mathbb{I}[c_{n,t+1} > \rho \vee m_{n,t+1} > \rho]. \quad (5)$$

The temporal model estimates

$$p_{n,t} = \Pr(y_{n,t+1} = 1 \mid \mathbf{h}_{n,t}), \quad (6)$$

where $\mathbf{h}_{n,t}$ contains current telemetry, two lags, first and second differences, three-sample means and ranges, short-horizon variation, and Q95 headroom features. A node passes the temporal screen when $p_{n,t} \leq \theta$, with θ selected only from calibration data.

3.3. Empirical pre-commit projection

The guardrail uses persistence residuals estimated on calibration transitions. Let $q_c^{0.95}$ and $q_m^{0.95}$ be the 95th-percentile absolute one-step residual margins for CPU and memory. The upper projections are

$$\hat{c}_{n,t+1}^{\text{up}} = c_{n,t} + q_c^{0.95}, \quad (7)$$

$$\hat{m}_{n,t+1}^{\text{up}} = m_{n,t} + q_m^{0.95}. \quad (8)$$

The guardrail utilization is

$$g_{n,t} = \max \left(\frac{\hat{c}_{n,t+1}^{\text{up}}}{\rho}, \frac{\hat{m}_{n,t+1}^{\text{up}}}{\rho} \right). \quad (9)$$

A destination is Q95-executable when $g_{n,t} \leq 1$. This is an empirical calibration rule. It is not a formal safety certificate and does not bound unmodelled actuation effects.

3.4. Feasible destination set

Let A_t be the set of active source nodes at timestamp t , $U_t(n)$ the number of destination reservations already assigned to node n at the same timestamp, and C the per-timestamp destination capacity. The feasible set for event e is

$$\mathcal{F}_e = \left\{ n \in \mathcal{N} : \begin{array}{l} L(s_e, n) = 1, \ p_{n,t} \leq \theta, \ g_{n,t} \leq 1, \\ n \notin A_t, \ U_t(n) < C \end{array} \right\}. \quad (10)$$

The source is retained when it satisfies legality, temporal risk, and Q95 conditions. Otherwise, the scheduler chooses a destination from $\mathcal{F}_e$. Excluding active source nodes prevents one simultaneous event from being migrated onto another event's source. Destination reservations prevent conflicting assignments.

3.5. Decision objective

The decision is lexicographic rather than a weighted sum. First, illegal and non-executable nodes are removed. Among feasible candidates, CAGE-S prefers nodes with lower historical selection counts and then lower temporal risk and guardrail utilization. For a shortlisted set $K_e \subseteq \mathcal{F}_e$, the selected destination is

$$n_e^* = \operatorname{lexmin}_{n \in K_e} (H_t(n), p_{n,t}, g_{n,t}, \operatorname{id}(n)), \quad (11)$$

where $H_t(n)$ is the cumulative number of prior selections. The fixed node identifier resolves exact ties. This objective deliberately avoids converting compliance or executability into penalties that can be traded against performance.

The evaluation does not assume that $y_{n_e^*, t+1}$ is the actual outcome after migration. It is the future state that the selected node naturally exhibited in the trace. We use it only as a common native next-state proxy for comparing schedulers under the same replay. Accordingly, the paper makes no causal claim about post-migration response time, resource interference, or achieved service-level improvement.

4. The CAGE-S scheduler

4.1. Overview

CAGE-S separates the scheduling path into four questions. *Is the node permitted?* is answered by the policy gate. *Is the node likely to cross the operational threshold in the next interval?* is answered by the temporal model. *Are repeated decisions being concentrated?* is handled by fair shortlisting. *Does the selected node still have empirical headroom immediately before commitment?* is checked by the Q95 projection. Keeping these questions separate makes the scheduler easier to inspect and prevents a strong score in one dimension from hiding a failure in another.

4.2. Deterministic policy gating

For every service class, the policy layer precomputes a Boolean mask over the ordered node list. A mask lookup is constant time per node and does not invoke the predictive model. In a real deployment, the mask can be generated from Kubernetes labels, admission-controller metadata, OPA decisions, remote-attestation status, or a policy registry. In the trace evaluation, the metadata are synthetically overlaid because the source trace contains no jurisdiction or sensitivity labels. Section 5.4 explains how the overlay is kept independent of telemetry and outcomes.

The scheduler evaluates the source first. An illegal source always requires migration. A legal source can still require migration when temporal risk exceeds θ or the Q95 projection exceeds one. This distinction supports three interpretable

trigger categories: policy-only, risk-only, and joint policy–risk triggers.

4.3. Temporal risk prediction

The predictor is a histogram gradient boosting classifier trained to estimate Eq. (6). Gradient-boosted trees were selected because the feature set is tabular, nonlinear interactions are expected, and inference must remain lightweight. The 22 features are listed in Appendix A. They fall into five groups:

- (a) current state and Q95 upper projections;
- (b) one- and two-step CPU and memory lags;
- (c) first and second temporal differences;
- (d) three-sample means and ranges;
- (e) short-horizon variation and headroom.

The model is trained only on calibration history. The threshold $\theta = 0.011362695$ is selected at a matched-availability operating point, not tuned on the test workload. A current-state-only HGB model is trained as a direct ablation. Because risk scores from different fitted models are not necessarily calibrated on the same numerical scale, scheduler comparisons use matched acceptance or matched risk conditions rather than comparing mean raw probabilities.

4.4. Fairness-aware shortlisting

Scoring every feasible node is unnecessary when the cluster contains more than twelve thousand nodes. At each timestamp, CAGE-S first orders legal candidates by cumulative selection count, with guardrail utilization used to stabilize the order, and retains at most $k = 256$ candidates. The shortlisted nodes are then filtered by temporal and Q95 conditions. If the first shortlist has no accepted node, the scheduler searches the remaining legal candidates before declaring infeasibility. In the evaluated workloads, no fallback was required.

The shortlist is not a heuristic added only for speed. It also controls concentration. A pure minimum-risk strategy can repeatedly select the same small group of nodes because many low-risk probabilities are close. Historical counts make the choice sensitive to repeated use while preserving risk as the secondary criterion. The value $k = 256$ was chosen after comparing $k \in \{128, 256, 512, 1024, 2048\}$ with full candidate scoring. The $k = 256$ configuration exactly reproduced full-fairness decisions in the validation study, had no fallback, and retained large minimum candidate slack.

4.5. Pre-commit guardrail and reservations

A destination that passes the temporal model can still fail the transparent Q95 projection because the two mechanisms capture different uncertainty. The temporal model learns nonlinear patterns in history. The guardrail adds calibrated residual margins to current CPU and memory and checks the resulting upper projection against the threshold. The

Algorithm 1: CAGE-S decision for one timestamp

Input: Events E_t ; ordered node snapshot X_t ; histories; policy masks; threshold θ ; shortlist size k ; destination capacity C

Output: A retained source or reserved legal destination for every event

```

1 Build active-source mask  $A_t$  and initialize destination usage  $U_t \leftarrow 0$ ;
2 Compute temporal risks  $p_{n,t}$  and Q95 guardrails  $g_{n,t}$  for all candidate nodes required at  $t$ ;
3 foreach  $e = (s_e, n_e, t) \in E_t$  in deterministic event order do
4    $sourceOK \leftarrow L(s_e, n_e) \wedge (p_{n_e,t} \leq \theta) \wedge (g_{n_e,t} \leq 1)$ ;
5   if  $sourceOK$  then
6      $n_e^* \leftarrow n_e$ ;
7   else
8      $B_e \leftarrow \{n : L(s_e, n) = 1, n \notin A_t, U_t(n) < C\}$ ;
9     Form a fairness-aware shortlist  $K_e$  of at most  $k$  nodes from  $B_e$ ;
10     $K_e \leftarrow \{n \in K_e : p_{n,t} \leq \theta, g_{n,t} \leq 1\}$ ;
11    if  $K_e = \emptyset$  then
12      search the remaining nodes in  $B_e$  and record fallback if used;
13     $n_e^* \leftarrow \text{lexmin}_{n \in K_e} (H_t(n), p_{n,t}, g_{n,t}, \text{id}(n))$ ;
14    Recheck  $L(s_e, n_e^*), p_{n_e^*,t} \leq \theta$ , and  $g_{n_e^*,t} \leq 1$ ;
15    Reserve the destination:  $U_t(n_e^*) \leftarrow U_t(n_e^*) + 1$ ;
16  Update historical selection count  $H_t(n_e^*)$  and emit the decision;
```

check is repeated after selection and before reservation. When multiple events share a timestamp, a destination usage counter is updated immediately so subsequent events see the reservation.

The default destination capacity is $C = 1$ per timestamp. A sensitivity run with $C = 2$ produced the same migration pattern, the same Jain fairness, and no timestamp at which a destination was used twice. This result indicates that $C = 1$ did not create an artificial bottleneck for the evaluated event density.

4.6. Complexity

Let M be the node count, $d = 22$ the feature dimension, R_t the number of nodes whose risks are inferred at timestamp t , and B_t the number of events. Policy masks and Q95 features require $O(M)$ preprocessing per timestamp. Histogram-gradient-boosting inference is approximately linear in the number of evaluated nodes and tree depth, $O(R_t D)$ for fixed tree structure. Fair shortlisting can be implemented with partial ordering in $O(M \log k)$, followed by $O(k)$ ranking per migrated event. In practice, inference is performed on a timestamp-level superset so per-event scoring does not repeatedly invoke the model. Section 6.6 reports measured end-to-end online-core latency.

5. Experimental design

5.1. Alibaba trace and cleaning

The evaluation uses the public Alibaba Microservices Trace 2021, which records node telemetry and microservice-instance resource signals at 30-second intervals (Luo et al., 2021). We process the full Node_0 archive and the MSResource_0 shard. The raw node archive contains

18,868,400 rows, 13,118 nodes, and 1,440 timestamps. Strict cleaning removes nodes with incomplete trajectories, leaving 18,803,520 rows for 13,058 nodes. The microservice shard contains 11,564,394 rows. After excluding nine service instances with missing CPU or memory and aligning to valid local-shard nodes, 11,561,072 rows, 1,303 services, 96,346 service-instance pairs, and 12,727 nodes remain.

Table 2 summarizes the datasets. We retain genuine values above one in the instance-level CPU field because they occur in the source trace and are not evidence of parsing error. These instance fields are used to detect bursts, not to claim absolute node-capacity consumption.

5.2. Event construction and temporal alignment

Natural workload arrivals are rare in this shard: 96,341 of 96,346 service-instance pairs are already present at the first timestamp. We therefore construct replay events from positive instance CPU bursts. After a 300-second warm-up, an event is emitted when the instance CPU increase exceeds 0.05. This yields 20,419 events. The final timestamp has no observable $t + 30$ -second outcome, so 95 events are removed, leaving 20,324 evaluable events over 108 timestamps.

Figure 2 shows the causal alignment. An event is detected from the instance change ending at t ; the scheduler observes node state and history at t ; the evaluation proxy is the node state at $t + 30$ seconds. No validation or test sample contributes to training labels or feature construction before its timestamp.

The natural pool is dominated by a small number of services: its ten most frequent services contribute approximately 46% of events. We therefore construct a balanced sensitivity workload by retaining at most two events per service and timestamp. The balanced test set contains 3,787 events and reduces the top-ten share to 22.55%.

Table 2

Trace preparation and replay datasets.

Dataset	Rows/events	Nodes	Services	Timestamps	Main use
Raw node trace	18,868,400	13,118	–	1,440	Integrity audit
Strict node trace	18,803,520	13,058	–	1,440	Historical node telemetry
Raw MSResource_0	11,564,394	12,731	1,303	120	Service-instance signals
Clean aligned MSResource_0	11,561,072	12,727	1,303	120	Event construction
Natural evaluable event pool	20,324	4,939 source nodes	368 event services	108	Main replay
Balanced cap-2 event pool	6,875	–	294/285 split services	108	Service-balance sensitivity
Natural test workload	11,629	–	–	59	Main reported comparison
Balanced test workload	3,787	–	–	59	Robustness comparison

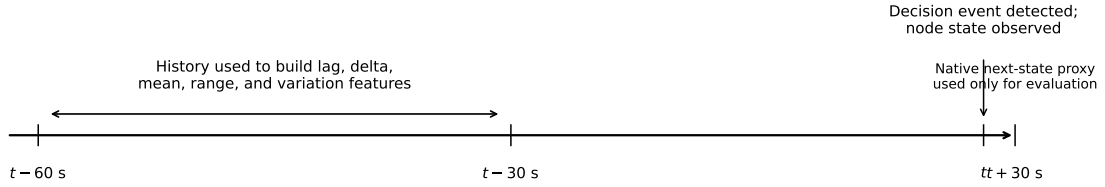


Figure 2: Leakage-controlled event alignment. The scheduler uses only current and historical observations. The state at $t + 30$ seconds is used as a native trace proxy for evaluation and is not interpreted as the counterfactual result of migration.

5.3. Calibration and test split

The temporal split point is 1,800,000 ms. Node transitions before the split form the calibration region; later transitions form the test region. Candidate model development uses a calibration-side development/validation split and never observes the final test region. Persistence residuals on calibration transitions give

$$q_c^{0.95} = 0.02592973, \quad q_m^{0.95} = 0.000219621. \quad (12)$$

On the node-transition test set, the joint Q95 rule accepts 82.11% of transitions and has an unsafe miss rate of 0.1343% among predicted-safe transitions. The temporal model threshold is selected on calibration data at the availability-matched operating point $\theta = 0.011362695$.

5.4. Policy overlay

The source trace contains no native policy labels. To evaluate scheduling semantics without pretending otherwise, node capabilities and service requirements are generated by stable hashing. The hash seed is independent of node telemetry, service outcomes, and placement. Nodes receive one of four jurisdictions (EU-West, EU-Central, US-East, and AP-Southeast), one of three trust levels, and Boolean encryption and TEE capabilities. Services receive one of six policy classes shown in Table 3.

Thirty policy seeds are used to vary the overlay. Across seeds, the natural test workload has a mean source-legality rate of 81.97% with a 5th–95th percentile range of 71.32–88.09%. The minimum number of legal destinations remains in the hundreds, and no seed produces an infeasible policy

Table 3

Policy classes in the deterministic overlay for the reference seed.

Service policy class	Legal nodes
Global public	12,727
Global protected	10,320
EU restricted	6,206
US restricted	2,613
APAC restricted	1,501
EU confidential	866

class. The overlay therefore tests policy selectivity without making legality artificially trivial or impossible.

5.5. Compared methods

All replay methods receive the same events, node ordering, policy metadata, concurrency rules, and outcome proxies. The compared methods are:

Observed source.

No scheduling action; the event remains at its native source.

Random-policy-Q95.

Randomly selects a legal Q95-accepted destination; results average 20 deterministic replicates.

Least-loaded-policy-Q95.

Chooses the legal candidate with the lowest current joint utilization and then lowest Q95 utilization.

Policy-only-fair.

Enforces policy and fair shortlisting but does not screen temporal or Q95 risk.

Policy-Q95-fair.

Uses policy, fair shortlisting, and Q95 feasibility without learned temporal risk.

Policy-boundary-0.99.

Uses a stricter deterministic guardrail cutoff of 0.99.

Policy-current-HGB-Q95.

Replaces temporal features with a current-state HGB risk model.

Temporal-Q95-no-policy.

Uses temporal prediction and Q95 but deliberately removes policy gating.

Policy-temporal-no-projection.

Removes only the final Q95 projection from CAGE-S.

CAGE-S.

Policy, temporal risk, fairness-aware $k = 256$ shortlist, reservation, and Q95 projection.

5.6. Metrics and statistical analysis

The primary metrics are policy-violation rate, temporal-threshold violation rate, Q95-violation rate, migration rate, native next-state unsafe proxy, destination Jain fairness, destination concentration, fallback rate, and decision latency. Jain fairness for migrated destinations is

$$J = \frac{(\sum_{n=1}^M z_n)^2}{M \sum_{n=1}^M z_n^2}, \quad (13)$$

where z_n is the number of migrations assigned to node n (Jain, Chiu and Hawe, 1984). Higher values indicate a more distributed assignment pattern.

For event-level rates, uncertainty is estimated with a paired timestamp-cluster bootstrap. A bootstrap sample resamples the 59 test timestamps with replacement and keeps all events within a sampled timestamp together. Random-baseline replicate identity is resampled inside each bootstrap iteration. We use 20,000 bootstrap replicates and report percentile 95% confidence intervals for the full-method-minus-baseline difference.

5.7. Implementation and timing scope

Preprocessing uses Polars, Apache Arrow, DuckDB, NumPy, SciPy, pandas, and scikit-learn (Pedregosa, Varoquaux, Gramfort et al., 2011). The online benchmark includes construction of all 22 features from three in-memory snapshots, source inference, policy filtering, fairness-aware shortlisting, destination inference, ranking, reservation, and pre-commit checks. It excludes telemetry collection, network and disk I/O, serialization, and Kubernetes actuation. Measurements were obtained under WSL2 on a Dell Precision 3561. [TBD: Insert exact CPU model, RAM, WSL2 version, and thread settings before submission.]

Table 4

Candidate node-risk prediction on the held-out test transitions.

Metric	Current-state HGB	Temporal HGB
ROC-AUC	0.996944	0.998248
PR-AUC	0.984894	0.990308
Brier score ↓	0.009858	0.008578
Log loss ↓	—	0.030480
Expected calibration error ↓	—	0.000970

6. Results**6.1. Does temporal history improve node-risk prediction?**

Table 4 compares the temporal and current-state HGB models. The temporal model improves every reported proper or ranking metric on the held-out test set. The gain in ROC-AUC is numerically small because both models already separate most safe and unsafe transitions, but the PR-AUC, Brier score, and scheduler-level experiments show that history remains useful in the rare-event operating region.

The temporal model's test ROC-AUC is 0.99825 and PR-AUC is 0.99031. More importantly, at the matched availability threshold, it accepts 82.35% of test node transitions with an unsafe rate of 0.0571%, while retaining 93.29% of actually safe transitions. This operating point becomes the default temporal screen.

6.2. End-to-end comparison on the natural workload

Table 5 reports the natural workload. The observed source is not a deployable compliance baseline: 25.87% of selected sources violate the synthetic policy overlay, 19.51% exceed the temporal threshold, and 19.45% fail the Q95 check. Policy-only scheduling removes policy violations but leaves 13.01% temporal and 13.02% Q95 violations, showing that legality and current availability cannot substitute for risk control.

Three comparisons explain the role of the complete method. First, relative to policy-current-HGB-Q95, CAGE-S reduces migration from 42.85% to 39.12%, removes the 0.370% temporal violation rate, and reduces the native unsafe proxy from 0.0688% to 0.0430%. Second, relative to policy-Q95-fair, it pays 0.92 percentage points of additional migration but removes a 0.980% temporal violation rate and reduces the proxy from 0.1548% to 0.0430%. Third, relative to policy-temporal-no-projection, the final guardrail costs 0.93 percentage points of migration and removes all 1.144% Q95 violations without changing the native proxy. The projection is therefore a commit-time backstop, not the source of the predictive gain.

6.3. Balanced workload

Table 6 shows the balanced workload. The pattern is consistent. CAGE-S produces zero policy, temporal, and Q95 violations with a 40.19% migration rate and 0.1056% native unsafe proxy. The current-state HGB baseline migrates more events (43.20%) while leaving 0.502% temporal violations.

Table 5

Natural test workload (11,629 events). Rates are percentages. Native unsafe is the selected node's observed state at $t + 30$ seconds and is not a counterfactual migration outcome.

Method	Migration	Policy viol.	Temporal viol.	Q95 viol.	Native unsafe	Migrated Jain	Top-10 migrated share
Observed source	0.000	25.875	19.512	19.451	12.340	0.000	0.000
Random-policy-Q95	38.198	0.000	1.392	0.000	0.190	0.191	1.353
Least-loaded-policy-Q95	38.198	0.000	0.920	0.000	0.155	0.009	13.282
Policy-only-fair	25.875	0.000	13.011	13.019	7.215	0.189	0.665
Policy-Q95-fair	38.198	0.000	0.980	0.000	0.155	0.291	0.450
Policy-boundary-0.99	43.323	0.000	0.404	0.000	0.095	0.332	0.397
Policy-current-HGB-Q95	42.850	0.000	0.370	0.000	0.069	0.329	0.401
Temporal-Q95-no-policy	20.741	25.411	0.000	0.000	0.060	0.190	0.415
Policy-temporal-no-projection	38.189	0.000	0.000	1.144	0.043	0.290	0.450
CAGE-S	39.118	0.000	0.000	0.000	0.043	0.299	0.440

Table 6

Balanced test workload (3,787 events). Rates are percentages.

Method	Migration	Policy viol.	Temporal viol.	Q95 viol.	Native unsafe	Migrated Jain	Top-10 migrated share
Observed source	0.000	24.611	21.917	21.310	13.890	0.000	0.000
Random-policy-Q95	38.791	0.000	1.863	0.000	0.334	0.094	2.155
Least-loaded-policy-Q95	38.791	0.000	1.400	0.000	0.290	0.002	39.959
Policy-only-fair	24.611	0.000	14.787	14.180	8.635	0.073	1.073
Policy-Q95-fair	38.791	0.000	1.400	0.000	0.290	0.115	0.681
Policy-boundary-0.99	43.623	0.000	0.528	0.000	0.158	0.130	0.605
Policy-current-HGB-Q95	43.200	0.000	0.502	0.000	0.158	0.129	0.611
Temporal-Q95-no-policy	23.105	24.399	0.000	0.000	0.132	0.069	1.143
Policy-temporal-no-projection	39.398	0.000	0.000	0.792	0.106	0.117	0.670
CAGE-S	40.190	0.000	0.000	0.000	0.106	0.120	0.657

The policy-Q95 baseline migrates slightly fewer events (38.79%) but has 1.400% temporal violations and a 0.2905% native proxy.

Figure 4 presents the migration-proxy trade-off. The figure should not be read as a causal Pareto frontier because the vertical axis is a native trace outcome. It nevertheless makes the internal comparison clear: the temporal variants occupy a low-proxy region, policy is necessary to remove illegal placements, and the projection changes migration and Q95 compliance rather than the proxy.

6.4. Paired bootstrap comparisons

Table 7 reports selected full-minus-baseline differences. A negative interval for a violation or proxy metric favours CAGE-S; a positive migration interval indicates the additional migration paid for stronger filtering. Relative to the current-HGB scheduler, CAGE-S significantly reduces migration and temporal violations on both workloads. The proxy reduction is directional and reaches the zero boundary of the percentile interval, so we do not describe it as unambiguously significant. Relative to policy-Q95, CAGE-S pays about 0.9–1.4 percentage points of migration while significantly reducing temporal violations and the proxy. Relative to the no-projection ablation, the extra migration exactly buys the removal of Q95 violations.

6.5. Policy-seed robustness

The complete policy-temporal-projection scheduler is repeated over 30 deterministic policy seeds. The experiment contains 462,480 event decisions. There are no duplicate

migrated destinations under capacity one, no migration into another active source, no selected policy violation, no temporal-threshold violation, no Q95 violation, no fallback, and no missing value.

For the natural workload, the mean migration rate is 35.23%, with a 5th–95th percentile range of 30.15–44.80%. The mean native unsafe proxy is 0.0496%, and migrated-destination Jain fairness is 0.3155. For the balanced workload, the corresponding values are 37.84%, 0.1056%, and 0.1126. Figure 5 visualizes migration sensitivity. The wider natural interval reflects stronger variation in service-policy assignments, not unstable model fitting; the predictive model remains fixed across seeds.

6.6. Decision latency

The optimized online core is fast enough for a 30-second control interval. On the natural workload it requires 20.25 ms on average, with P95 32.08 ms and P99 57.51 ms. On the balanced workload it requires 13.15 ms on average, with P95 16.23 ms and P99 17.43 ms. The natural workload is slower because more events require migration and the policy-constrained inference superset is larger.

The benchmark validates feature equivalence against the offline snapshots to machine precision: the maximum feature difference is 2.22×10^{-16} and the maximum probability difference is zero. No online decision differs from the stored replay reference. The $k = 256$ shortlist itself requires approximately 6.01 ms on average in the natural workload when policy-gated superset inference is excluded.

Table 7

Selected paired timestamp-cluster bootstrap results. Differences are CAGE-S minus baseline; intervals are 95% percentile confidence intervals. All values are proportions.

Workload	Baseline and metric	Difference	CI low	CI high
Natural	Current-HGB: migration	-0.037320	-0.040806	-0.033591
Natural	Current-HGB: temporal violation	-0.003698	-0.005049	-0.002499
Natural	Current-HGB: native unsafe proxy	-0.000258	-0.000574	0.000000
Natural	Policy-Q95: migration	0.009201	0.007527	0.010939
Natural	Policy-Q95: temporal violation	-0.009803	-0.011684	-0.008022
Natural	Policy-Q95: native unsafe proxy	-0.001118	-0.001751	-0.000571
Natural	No projection: migration	0.009287	0.007350	0.011482
Natural	No projection: Q95 violation	-0.011437	-0.013933	-0.009146
Balanced	Current-HGB: migration	-0.030103	-0.035847	-0.024455
Balanced	Current-HGB: temporal violation	-0.005017	-0.007339	-0.002936
Balanced	Policy-Q95: migration	0.013995	0.010437	0.017647
Balanced	Policy-Q95: temporal violation	-0.013995	-0.017647	-0.010437
Balanced	Policy-Q95: native unsafe proxy	-0.001848	-0.003484	-0.000528
Balanced	No projection: migration	0.007922	0.005424	0.010571
Balanced	No projection: Q95 violation	-0.007922	-0.010571	-0.005424

6.7. Capacity sensitivity

Allowing two destination assignments per timestamp does not change the migration pattern. In the balanced workload, every destination still receives at most one assignment. In the natural workload, only 12 of 11,629 selected node identifiers differ, the migration indicators remain identical, and no destination is used twice. Jain fairness is unchanged. We therefore retain capacity one as the conservative main setting and report capacity two as a sensitivity result rather than an alternative method.

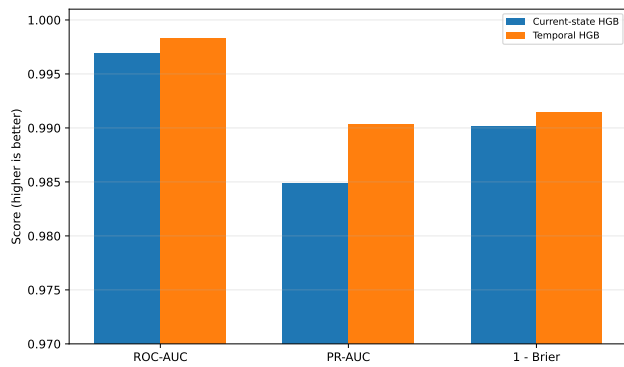


Figure 3: Held-out model quality. The plotted Brier quantity is $1 - \text{Brier}$ so that higher values are consistently better. Scheduler comparisons use matched operating points rather than raw probability averages.

Compliance-gated temporal-risk scheduling

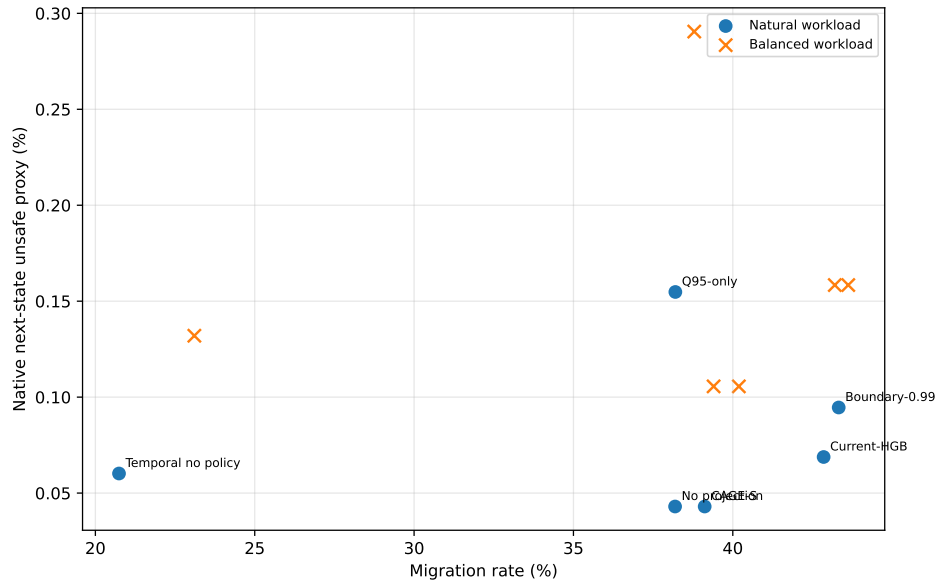


Figure 4: Migration versus native next-state unsafe proxy for representative methods. Lower is better on both axes, but the proxy is not the post-migration causal state.

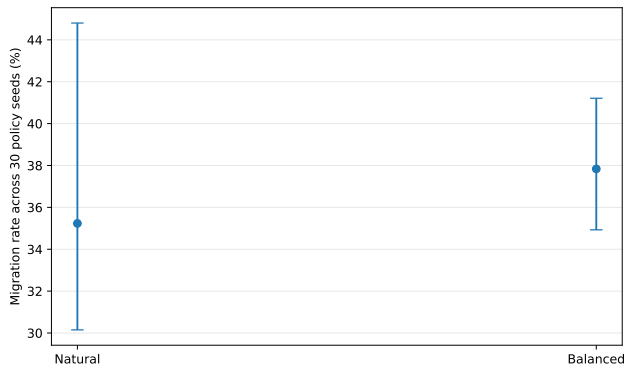


Figure 5: Mean and 5th–95th percentile migration rate over 30 deterministic policy overlays. All seeded runs retain zero selected policy, temporal, and Q95 violations.

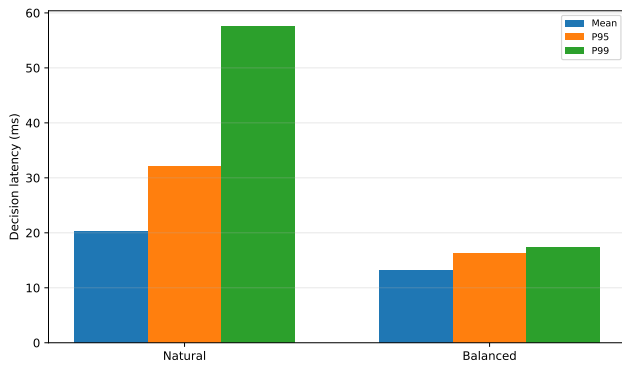


Figure 6: Measured latency of the complete in-memory online core. Telemetry transport, network and disk I/O, serialization, and orchestration actuation are outside the scope of this trace-driven timing study.

7. Discussion

7.1. What is new?

The individual tools used by CAGE-S are familiar: policy masks, gradient-boosted trees, quantile margins, and Jain fairness are established techniques. The contribution lies in how they are assigned distinct responsibilities in an online scheduler. Existing network-aware systems optimize communication or response time; policy frameworks enforce placement rules; temporal resource managers predict or react to SLO problems. CAGE-S places these components in a strict order that reflects the semantics of sensitive workloads.

First, legality is not a penalty. An illegal node is absent from the candidate set, so no performance score can compensate for a policy failure. Second, prediction is temporal and evaluated at the scheduler level. Current telemetry is a strong baseline, but the experiments show that history reduces migration and temporal violations under matched conditions. Third, fairness is implemented before final ranking rather than reported only after the fact. Fourth, the empirical projection is applied at commit time and is evaluated through the no-projection ablation. The result is a transparent pipeline in which every migration can be explained as policy-only, risk-only, projection-only, or a combination.

7.2. Interpretation of the trade-off

The complete method does not dominate every baseline on every metric. Against policy-Q95, it performs slightly more migrations to remove temporal violations and reduce the native unsafe proxy. Against a stricter 0.99 boundary and the current-HGB baseline, it performs fewer migrations while improving temporal compliance. Against the no-projection ablation, it performs about one percentage point more migrations to eliminate Q95 violations, with no change in the native proxy. These are useful and defensible trade-offs. Claiming universal superiority would hide the operational choice between migration cost and risk tolerance.

The fairness results also require context. Least-loaded placement has very low Jain fairness because it repeatedly chooses the same apparent minimum. Random placement is more distributed but has worse temporal violations and higher proxy risk. The fair shortlist improves distribution without making randomness the objective. The balanced workload has lower absolute Jain values because it contains fewer events and destinations, so cross-workload fairness values should not be compared as if sample sizes were identical.

7.3. Threats to validity

Construct validity. The policy metadata are synthetic. They test whether the scheduler obeys hard rules and remains feasible across policy distributions; they do not describe the real regulatory distribution of Alibaba workloads. The native next-state unsafe metric is also a proxy. It indicates what happened naturally to the selected node, not what would have happened after receiving a migrated service.

Internal validity. Temporal leakage is controlled by constructing labels, thresholds, residual margins, and model parameters only from past or calibration data. Event detection

uses the instance burst ending at t , node state is observed at t , and outcome is read at $t + 30$ seconds. The final timestamp is excluded. Deterministic seeds and event ordering make the replay reproducible.

External validity. The evaluation uses one microservice-resource shard from one trace provider. Production clusters may have different telemetry scales, admission rules, actuation delays, and service graphs. The results therefore establish performance under the stated trace and replay protocol rather than universal generalization across platforms. The WSL2 timing study measures algorithmic overhead only; it does not represent end-to-end control-plane latency or migration completion time.

Model validity. Histogram gradient boosting performs well on this trace, but concept drift can change calibration. A deployment should monitor Brier score, calibration error, acceptance rate, and threshold-crossing misses, and should retrain only from temporally valid data. The empirical Q95 guardrail is not a proof of safety and does not account for candidate-specific interference, network staging, thermal constraints, or the transient old-plus-new resource demand of rolling migration.

7.4. Practical deployment

A Kubernetes implementation can map the four layers to familiar extension points. An admission controller or scheduler filter plugin enforces policy. A node-telemetry service maintains the three-snapshot feature state and risk scores. The shortlist and ranking execute in a scheduler score plugin or external scheduler. A reserve or pre-bind hook repeats the policy, temporal, and Q95 checks and records the destination reservation. If the recheck fails, the scheduler selects the next candidate or emits an explicit infeasibility decision. This structure avoids silently falling back to an illegal node.

Policy changes and attestation revocation require immediate mask updates. A service whose current source becomes illegal should be migrated even if its operational risk is low. Conversely, a temporary load spike should not relax jurisdiction or TEE requirements. The separation of policy and risk makes these cases straightforward.

8. Conclusion

This paper introduced CAGE-S, a compliance-gated online scheduler for sensitive edge-cloud microservices. The method treats legality as a hard prerequisite, predicts next-interval operational risk from short telemetry history, distributes repeated choices through fair shortlisting, and rechecks empirical Q95 headroom before commitment. The design is intentionally modular: policy determines where a service may run, temporal prediction identifies fragile legal nodes, fairness limits concentration, and the guardrail verifies the selected action.

On the Alibaba Microservices Trace 2021, the temporal model improves discrimination and calibration over a current-state model. In the natural workload, the complete scheduler produces zero selected policy, temporal, and Q95 violations

Table 8

Features used by the temporal histogram-gradient-boosting model.

No.	Feature	Interpretation
1	observed_cpu_at_decision	Current node CPU signal
2	observed_memory_at_decision	Current node memory signal
3	upper_cpu_q95	Persistence CPU projection plus calibration margin
4	upper_memory_q95	Persistence memory projection plus calibration margin
5	current_joint_utilisation	Maximum normalized current CPU/memory utilization
6	guardrail_utilisation	Maximum normalized Q95 projection
7	cpu_headroom_q95	CPU threshold minus Q95 projection
8	memory_headroom_q95	Memory threshold minus Q95 projection
9–10	cpu_lag_1, cpu_lag_2	One- and two-step CPU history
11–12	memory_lag_1, memory_lag_2	One- and two-step memory history
13–14	cpu_delta_1, cpu_delta_2	Recent CPU changes
15–16	memory_delta_1, memory_delta_2	Recent memory changes
17–18	cpu_mean_3, memory_mean_3	Three-snapshot mean
19–20	cpu_range_3, memory_range_3	Three-snapshot range
21–22	cpu_variation_2, memory_variation_2	Short-horizon variation

with a 39.12% migration rate and a 0.043% native next-state unsafe proxy. The balanced workload shows the same qualitative pattern. Thirty policy seeds covering 462,480 decisions produce no policy violation, temporal-threshold violation, Q95 violation, destination conflict, fallback, or missing value. The complete in-memory decision core remains well below the 30-second control period.

The results support the central claim that sensitive scheduling should not collapse legality, operational prediction, and executability into one weighted score. They do not establish a causal post-migration response-time benefit, end-to-end orchestration latency, or universal generalization beyond the evaluated trace shard. Those limits are explicit: the contribution of this paper is a leakage-controlled, trace-driven evaluation of a legality-first scheduling pipeline and its operational trade-offs, not a claim of measured live-cluster improvement.

A. Temporal feature definition

The 22 temporal-model features are listed in Table 8. Features are constructed independently for each node from the current and two preceding aligned snapshots.

B. Reproducibility checkpoints

The principal immutable artifacts used in the current draft are:

- strict node parquet SHA-256: bf8021e9142e2c831995b677ae1ec5bb73e9f0a0a065caa6103807bb8dfee389;
- clean MSResource_0 parquet SHA-256: af666449cf1fbd17858dcc761f607806bf1695b50d8de72ba0f97dfe84d2966c;
- aligned node snapshot SHA-256: 418989653df6c754d80b82caa4b1845064d5395315aad76cba82771d486811c;
- temporal-risk snapshots SHA-256: 3f32477705d2986b9ae68fe9383d8e13c9d73e0335966a31be37f63ce8e6b83f;

- final baseline assignments SHA-256: 005b96606d32e74b7cb6427bb78134839146ab4b2797658fc11ab48ed41a451b;
- paired-bootstrap output SHA-256: f19f69cb2482e1aeeabb664b473d2c1749768449ee89531c42ab34fcc6ca36b9.

C. Checklist for removing draft placeholders

Before submission, search the project for the token TBD. Every occurrence must be resolved. At minimum, the final version must contain exact author affiliations and institutional emails, exact timing hardware and thread settings, an anonymous artifact link or archival DOI, and final CRediT roles. No placeholder should be replaced with an estimated value.

Data and code availability

The Alibaba Microservices Trace 2021 is publicly available through the Alibaba Cluster Trace Program. The complete preprocessing, model-training, trace-replay, bootstrap, and figure-generation artifact will be released upon acceptance. **[TBD: Insert the final anonymous artifact URL and archival DOI before submission.]**

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Omitted for anonymous review.

References

- Chen, M., Islam, M.T., Rodriguez Read, M., Buyya, R., 2026. TraDE: Network and traffic-aware adaptive scheduling for microservices under dynamics. *IEEE Transactions on Parallel and Distributed Systems* 37.

- Gan, Y., Liang, M., Dev, S., Lo, D., Delimitrou, C., 2021. Sage: Practical and scalable ml-driven performance debugging in microservices, in: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ACM. pp. 135–151.
- Gan, Y., Zhang, Y., Cheng, D., et al., 2019. An open-source benchmark suite for microservices and their hardware–software implications for cloud and edge systems, in: Proceedings of the 24th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ACM. pp. 3–18.
- Jain, R., Chiu, D.M., Hawe, W., 1984. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. DEC Research Report TR-301.
- Kannan, R.S., Subramanian, L., Raju, A., Ahn, J., Mars, J., Tang, L., 2019. GrandSLAM: Guaranteeing slas for jobs in microservices execution frameworks, in: Proceedings of the Fourteenth EuroSys Conference, ACM.
- Kubernetes Authors, 2026. Kubernetes documentation: Scheduling, eviction, and assigning pods to nodes. <https://kubernetes.io/docs/>. Accessed 12 July 2026.
- Liu, J., Zhang, S., Wang, Q., 2023. ConAdapter: Reinforcement learning-based fast concurrency adaptation for microservices in cloud, in: Proceedings of the ACM Symposium on Cloud Computing, ACM. pp. 427–442.
- Luo, S., Lin, C., Ye, K., Xu, G., Zhang, L., Yang, G., Xu, H., Xu, C., 2024. Optimizing resource management for shared microservices: A scalable system design. *ACM Transactions on Computer Systems* 42.
- Luo, S., Xu, H., Lu, C., Ye, K., Xu, G., Zhang, L., Ding, Y., He, J., Xu, C., 2021. Characterizing microservice dependency and performance: Alibaba trace analysis, in: Proceedings of the ACM Symposium on Cloud Computing, ACM. pp. 412–426.
- Marchese, A., Tomarchio, O., 2022a. Extending the kubernetes platform with network-aware scheduling capabilities, in: International Conference on Service-Oriented Computing, Springer. pp. 465–480.
- Marchese, A., Tomarchio, O., 2022b. Network-aware container placement in cloud–edge kubernetes clusters, in: IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing, pp. 859–865.
- Mirhosseini, A., Elnikety, S., Wensch, T.F., 2021. Parslo: A gradient descent-based approach for near-optimal partial slo allotment in microservices, in: Proceedings of the ACM Symposium on Cloud Computing, ACM. pp. 442–457.
- Pallewatta, S., Babar, M.A., 2024. Towards secure management of edge–cloud iot microservices using policy as code, in: European Conference on Software Architecture (ECSA), Springer.
- Pallewatta, S., Kostakos, V., Buyya, R., 2023. Placement of microservices-based iot applications in fog computing: A taxonomy and future directions. *ACM Computing Surveys* 55.
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al., 2011. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research* 12, 2825–2830.
- Qiu, H., Banerjee, S.S., Jha, S., Kalbarczyk, Z.T., Iyer, R.K., 2020. FIRM: An intelligent fine-grained resource management framework for slo-oriented microservices, in: 14th USENIX Symposium on Operating Systems Design and Implementation, USENIX Association. pp. 805–825.
- Shen, S., Han, Y., Wang, X., Wang, S., Leung, V.C.M., 2023. Collaborative learning-based scheduling for kubernetes-oriented edge–cloud networks. *IEEE/ACM Transactions on Networking* doi:10.1109/TNET.2023.3267168.
- Song, C., Xu, M., Ye, K., Wu, H., Gill, S.S., Buyya, R., Xu, C., 2023. ChainsFormer: A chain latency-aware resource provisioning approach for microservices cluster, in: International Conference on Service-Oriented Computing, Springer. pp. 197–211.
- Thijsman, J., Sebrechts, M., De Turck, F., Volckaert, B., 2024. Trusting the cloud-native edge: Remotely attested kubernetes workers. *arXiv:2405.10131*.
- Wen, L., Xu, M., Gill, S.S., Hilman, M.H., Srirama, S.N., Ye, K., Xu, C., 2024. StatuScale: Status-aware and elastic scaling strategy for microservice applications. *arXiv:2407.10173*. preprint; journal publication metadata to be updated in the final bibliography.
- Wojciechowski, L., Opasiak, K., Latusek, J., Wereski, M., Morales, V., Kim, T., Hong, M., 2021. NetMARKS: Network metrics-aware kubernetes scheduler powered by service mesh, in: IEEE INFOCOM, IEEE. pp. 1–9.
- Zhu, X., Zhu, X., et al., 2023. On optimizing traffic scheduling for multi-replica containerized microservices, in: Proceedings of the 52nd International Conference on Parallel Processing, ACM. pp. 358–368.